Sample Solution for Homework 9

Problem 1 Objects and Subtyping (16 Points)

(a) Indicate whether the following JakartaScript subtype relationships are true or false.

(i) `Num <: Num` Answer: true

(ii) `Bool <: Num` Answer: false

(iii) `{let f: Num} <: {let f: Any}` Answer: false

(iv) `{let f: Num} <: {const f: Any}` Answer: true

(v) `{const f: Num} <: {let f: Num}` Answer: false

(vi) `{let f: Num} <: {let g: Num}` Answer: false

(vii) `{const f: {}} <: {const f: Any, let g: Bool}` Answer: false

(viii) `Any => Bool <: Bool => Any` Answer: true

(ix) `Bool => Bool <: Any => Any` Answer: false

(b) For each of the following programs: (1) say whether the program can be safely evaluated or not, i.e., whether the evaluation will get stuck or produce a value. If it can be safely evaluated, provide the computed value; (2) determine whether the program is well-typed with subtyping (i.e., with the rule TypeSub). If it is not, provide a brief explanation.

(i)

```
const x = {let f: 3};
const fun = function(y: {let f: Num, const g: Bool}) {
    y.f = 4; return y;
  };
fun(x).f
```

The program can be safely evaluated. The result value is 4. The program is not well-typed. The type error is at the function call `fun(x)` because the inferred type for `x` is `{let f: Num}`, which is not a subtype of the type of the parameter `y` of `fun`.

(ii)

```
const x = {let f: 3, const g: true};
const fun = function(y: {let f: Num}) {
    return y;
  };
fun(x).f
```

The program can be safely evaluated. The result value is 3. The program is well-typed. The inferred type is `Num`.

(iii)

```
const x = {let f: 3, const g: true};
const fun = function(y: {let f: Num}) {
    return y;
  };
fun(x).g
```

The program can be safely evaluated. The result value is **true**. The program is not well-typed. The problem is that the program dereferences field `g` of the object returned by `fun(x)`. However, the inferred return type of function `fun` is `{let f: Num}`, which has no field `g`.

(iv)

```
const x = {let f: 1, const g: true};
const y = {const f: false, let g: 2};
const z = true ? x : y;
z.f
```

The program can be safely evaluated. The result value is 1. The program is well-typed. The inferred type is **Any**.